

views or a form. Then test each method for defined functionality tests.

The second way is to create a separate class for different use cases. So, the test functions are validated based on the definitions of models.

The entire structure, however, depends on the business requirements. You can customise the design of tests according to the functions in the web application. Separate test classes can also be created for models, views, and forms based on an object's specific functionality in the Django app.

Here is an example of adding test case files that demonstrate how TestCase can help structure the entire test code.

```

• class YourTestClass(TestCase):
•     @classmethod
•     def setUpTestData(cls):
•         print("setUpTestData: Run once to set up non-mod-
•             ified data for all class methods.")
•         pass
•
•     def setUp(self):
•         print("setUp: Run once for every test method to
•             setup clean data.")
•         pass
•
•     def test_false_is_false(self):
•         print("Method: test_false_is_false.")
•         self.assertFalse(False)
•
•     def test_false_is_true(self):
•         print("Method: test_false_is_true.")
•         self.assertTrue(False)
•
•     def test_one_plus_one_equals_two(self):
•         print("Method: test_one_plus_one_equals _
•             two.")
•         self.assertEqual(1 + 1, 2)

```

Once you create the basic test structure, you will need to define the new classes. There are two methods to do so, and the methods that can help in the pre-configuration of the test run are as follows: